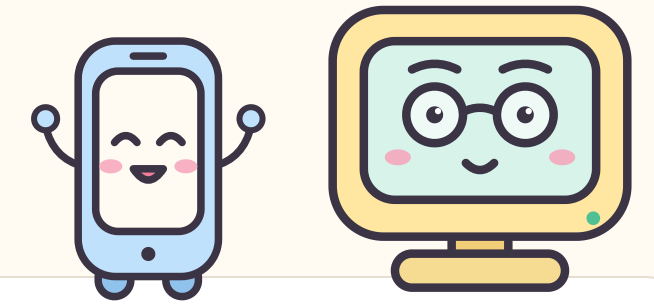




第 16 集 · 约 23 分钟

桌面①：程序入口

双击之后的那一秒钟，老电脑做了哪些事



从零写出 LanShare

这一集你会学到

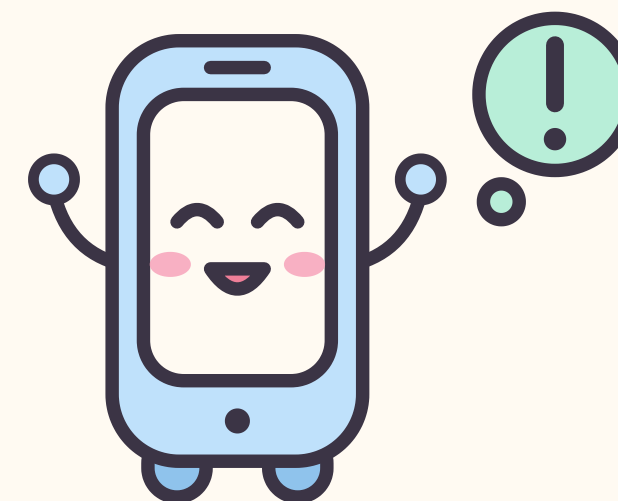
1. 程序入口的全流程：参数、单实例、绑端口、起服务、开窗口、托盘
2. 手动建 tokio 运行时：主线程为什么要留给托盘
3. SO_EXCLUSIVEADDRUSE：别让别人的程序和我们“共用”端口
4. build.rs：编译之前先跑一段小程序，把图标编进 exe
5. 先放一个“替身”，让上层代码先跑起来



先想一个问题

双击 LanShare.exe 之后，到托盘里出现那个小图标，这一秒钟里老电脑做了哪些事？

接上控制台、读参数、确认自己是不是唯一的那一个、绑端口、起服务、打开主界面，最后把主线程交给托盘。





启动的全流程





端口劫持：两个程序“共用”一个端口





加依赖：socket2, windows-sys 再开一个模块

● ● ●

Cargo.toml

crates/lanshare/

第 14-33 行 • 共 33 行

```
14  base64 = "0.22"
15  dirs = "6"
16  hex = "0.4"
17  hyper = { version = "1", features = ["http1", "server"] }
18  hyper-util = { version = "0.1.21", features = ["tokio", "server-graceful"] }
19  if-addr = "0.13"
20  lanshare-proto = { path = "../lanshare-proto" }
21  qrcode = { version = "0.14", default-features = false, features = ["svg"] }
22  rand_core = { version = "0.6", features = ["getrandom"] }
23  serde = { version = "1", features = ["derive"] }
24  serde_json = "1"
25  socket2 = "0.6.5"
26  tokio = { version = "1", features = ["rt-multi-thread", "macros", "net", "fs", "io-util", "sync", "time", "signal"] }
27  tower = { version = "0.5", features = ["util"] }
28  windows-sys = { version = "0.61.2", features = ["Win32_System_SystemInformation", "Win32_Storage_FileSystem", "Win32_Networking_WinSock"] } 临时
29
30  [dev-dependencies]
31  request = { version = "0.12", default-features = false, features = ["json"] }
32  sha2 = "0.10"
33  tempfile = "3"
```



net.rs: 独占地绑定端口

```
net.rs crates/lanshare/src/ 第 82-101 行 • 共 101 行

82     let second = bind_with_fallback(Ipv4Addr::UNSPECIFIED, p).unwrap();
83     assert_ne!(port_of(&second), p, "不能和第一个实例共用端口");
84 }
85
86 #[test]
87 fn port_taken_on_a_specific_address_is_skipped() {
88     // 别的程序 (比如 Django runserver) 只占了 127.0.0.1:P — 用的是普通绑定, 没有独占
89     let blocker = std::net::TcpListener::bind((Ipv4Addr::LOCALHOST, 0)).unwrap();
90     let p = port_of(&blocker);
91     let ours = bind_with_fallback(Ipv4Addr::UNSPECIFIED, p).unwrap();
92     assert_ne!(port_of(&ours), p, "0.0.0.0:P 不能和别人的 127.0.0.1:P 共存");
93 }
94
95 #[test]
96 fn listener_is_non_blocking_for_tokio() {
97     let l = bind_with_fallback(Ipv4Addr::LOCALHOST, 0).unwrap();
98     let err = l.accept().unwrap_err();
99     assert_eq!(err.kind(), io::ErrorKind::WouldBlock);
100 }
101 }
```



登记 net 和 desktop

● ● ●

lib.rs

crates/lanshare/src/

第 1-10 行 • 共 10 行

```
1  //! 局域网快传 v2: 服务端与桌面外壳的库部分 (`main.rs` 只负责启动)。  
2  
3  pub mod cli;  
4  pub mod desktop;  
5  pub mod logging;  
6  pub mod net;  
7  pub mod netinfo;  
8  pub mod server;  
9  pub mod storage;  
10 pub mod textboard;
```



先放一个替身：Python 和 Rust

Python • 测试里的替身

```
1 class FakeShell:
2     def show_error(self, message):
3         print(message)          # 真的会弹窗，替身只打印
4
5     def open_app_window(self, url):
6         pass                    # 真的会开浏览器，替身什么都不做
```

Rust • 临时的 desktop 模块

```
1 pub mod shell {
2     pub fn show_error(message: &str) { eprintln!("{message}") }
3     pub fn open_app_window(_: &str) {}
4     // .....名字和参数都和真的一样
5 }
```



desktop/mod.rs: 临时替身

mod.rs

crates/lanshare/src/desktop/ 临时

第 8-27 行 · 共 27 行

```

8     pub fn write_record(_: &std::path::Path, _: &InstanceRecord) -> std::io::Result<()> { Ok(()) } 临时
9     pub fn read_record(_: &std::path::Path) -> Option<InstanceRecord> { None } 临时
10    pub fn remove_record_if_own(_: &std::path::Path, _: u32) {} 临时
11 } 临时
12 pub mod shell { 临时
13     use std::path::Path; 临时
14     pub fn attach_parent_console() -> bool { true } 临时
15     pub fn show_error(message: &str) { eprintln!("{message}") } 临时
16     pub fn open_app_window(_: &str) {} 临时
17     pub fn open_folder(_: &Path) {} 临时
18     pub struct Explorer; 临时
19     impl crate::server::Desktop for Explorer { 临时
20         fn reveal(&self, _: &Path) {} 临时
21         fn open_folder(&self, _: &Path) {} 临时
22     } 临时
23 } 临时
24 pub mod tray { 临时
25     pub type Action = std::sync::Arc<dyn Fn() + Send + Sync>; 临时
26     pub fn run(_: &str, _: Action, _: Action) -> Result<(), String> { Err("托盘在第 17 集写, 先加 --no-tray".into()) } 临时
27 } 临时

```



build.rs: 编译之前先跑的小程序

● ● ●

build.rs crates/lanshare/

第 1-15 行 • 共 15 行

```
1  //! 给 exe 嵌入图标和版本信息（资源管理器、任务栏、托盘都用这个图标）。
2
3  fn main() {
4      println!("cargo:rerun-if-changed=web/icon.ico");
5      if std::env::var("CARGO_CFG_TARGET_OS").as_deref() == Ok("windows") {
6          let mut res = winresource::WindowsResource::new();
7          res.set_icon("web/icon.ico"); // 资源编号 1, 托盘图标从这里读
8          res.set("FileDescription", "LanShare - LAN file transfer (end-to-end encrypted)");
9          res.set("ProductName", "LanShare");
10         res.set("OriginalFilename", "LanShare.exe");
11         if let Err(e) = res.compile() {
12             println!("cargo:warning=嵌入图标失败: {e}");
13         }
14     }
15 }
```



构建依赖：winresource



Cargo.toml

crates/lanshare/

第 17-36 行 • 共 36 行

```
17 hyper = { version = "1", features = ["http1", "server"] }
18 hyper-util = { version = "0.1.21", features = ["tokio", "server-graceful"] }
19 if-addr = "0.13"
20 lanshare-proto = { path = "../lanshare-proto" }
21 qrcode = { version = "0.14", default-features = false, features = ["svg"] }
22 rand_core = { version = "0.6", features = ["getrandom"] }
23 serde = { version = "1", features = ["derive"] }
24 serde_json = "1"
25 socket2 = "0.6.5"
26 tokio = { version = "1", features = ["rt-multi-thread", "macros", "net", "fs", "io-util", "sync", "time", "signal"] }
27 tower = { version = "0.5", features = ["util"] }
28 windows-sys = { version = "0.61.2", features = ["Win32_System_SystemInformation", "Win32_Storage_FileSystem", "Win32_Networking_WinSock"] }
29
30 [dev-dependencies]
31 request = { version = "0.12", default-features = false, features = ["json"] }
32 sha2 = "0.10"
33 tempfile = "3"
34
35 [build-dependencies]
36 winresource = "0.1.31"
```

临时





main.rs: 开头

● ● ●

main.rs

crates/lanshare/src/

第 10-29 行 • 共 29 行

```
10 use std::time::{Duration, Instant};
11
12 use lanshare::cli::{self, Args, CliError};
13 use lanshare::desktop::instance::{self, InstanceLock, InstanceRecord};
14 use lanshare::desktop::shell;
15 use lanshare::logging::FileLog;
16 use lanshare::net;
17 use lanshare::netinfo;
18 use lanshare::server::{self, AppState, Config};
19
20 const MUTEX_NAME: &str = "Local\\LanShare-v2";
21
22 /// 有控制台就打印，没有就弹窗。
23 fn report(has_console: bool, message: &str) {
24     if has_console {
25         eprintln!("{message}");
26     } else {
27         shell::show_error(message);
28     }
29 }
```



main.rs: main 函数

main.rs crates/lanshare/src/ 第 56-75 行 • 共 75 行

```
56         }
57         None => return hand_over_to_running(&record_path, &args, &log, has_console),
58     }
59 } else {
60     None
61 };
62
63 match run(&args, &log, &record_path) {
64     Ok(()) => {
65         log.write("已退出");
66         ExitCode::SUCCESS
67     }
68     Err(message) => {
69         log.write(&message);
70         // --no-tray 是给终端和自动化测试用的：不弹窗，免得卡住测试
71         report(has_console || args.no_tray, &message);
72         ExitCode::from(1)
73     }
74 }
75 }
```



main.rs: run, 开张之前

```
main.rs crates/lanshare/src/ 第 94-113 行 • 共 113 行

94     if args.single_instance {
95         let record = InstanceRecord { port, key: state.key_b64(), pid: std::process::id() };
96         instance::write_record(record_path, &record).map_err(|e| format!("写实例记录失败: {e}"))?;
97     }
98     if let Some(path) = &args.info_file {
99         let info = serde_json::json!({
100             "port": port,
101             "pin": state.pin(),
102             "key": state.key_b64(),
103             "server_id": state.server_id(),
104             "urls": state.base_urls(),
105         });
106         std::fs::write(path, info.to_string()).map_err(|e| format!("写 info 文件失败: {e}"))?;
107     }
108     log.write(&format!(
109         "局域网快传 v{} 已启动: {} (共享文件夹 {})",
110         env!("CARGO_PKG_VERSION"),
111         state.base_urls()[0],
112         state.share_dir().display()
113     ));
```



main.rs: 手动建运行时，主线程留给托盘

```
main.rs crates/lanshare/src/ 第 111-130 行 • 共 130 行

111     state.base_urls()[0],
112     state.share_dir().display()
113 ));
114
115 let runtime = tokio::runtime::Builder::new_multi_thread()
116     .enable_all()
117     .build()
118     .map_err(|e| format!("创建运行时失败: {e}"))?;
119 let listener = {
120     let _guard = runtime.enter(); // from_std 必须在运行时里调用
121     tokio::net::TcpListener::from_std(listener).map_err(|e| format!("监听失败: {e}"))?
122 };
123 let (stop_tx, stop_rx) = tokio::sync::oneshot::channel::<()>();
124 let server_task = runtime.spawn(server::serve_with_shutdown(listener, state.clone(), async {
125     let _ = stop_rx.await;
126 })));
127
128 if !args.no_browser {
129     shell::open_app_window(&state.local_entry_url());
130 }
```



main.rs：前台运行，还是托盘

● ● ●

main.rs

crates/lanshare/src/

第 137–156 行 • 共 156 行

```
137         r = server_task => match r {
138             Ok(Ok(())) => Ok(()),
139             Ok(Err(e)) => Err(format!("服务出错: {e}")),
140             Err(e) => Err(format!("服务异常退出: {e}")),
141         },
142     }
143 })
144 } else {
145     let tray = run_tray(&state);
146     let _ = stop_tx.send(());
147     runtime.block_on(async {
148         let _ = tokio::time::timeout(Duration::from_secs(3), server_task).await;
149     });
150     tray
151 };
152 if args.single_instance {
153     instance::remove_record_if_own(record_path, std::process::id());
154 }
155 result
156 }
```



main.rs: 托盘和交接

```
main.rs crates/lanshare/src/ 第 171-190 行 • 共 190 行

171 fn run_tray(_state: &Arc<AppState>) -> Result<(), String> {
172     Err("托盘只在 Windows 上可用, 请加 --no-tray".to_string())
173 }
174
175 /// 已经有实例在运行: 等它写好实例记录 (它可能刚启动), 然后打开它的主界面。
176 fn hand_over_to_running(record_path: &Path, args: &Args, log: &FileLog, has_console: bool) -> ExitCode {
177     let deadline = Instant::now() + Duration::from_secs(10);
178     while Instant::now() < deadline {
179         if let Some(record) = instance::read_record(record_path) {
180             if !args.no_browser {
181                 shell::open_app_window(&format!("http://127.0.0.1:{}", record.port, record.key));
182             }
183             log.write("已经在运行, 打开已有实例的主界面");
184             return ExitCode::SUCCESS;
185         }
186         std::thread::sleep(Duration::from_millis(100));
187     }
188     report(has_console, "局域网快传已经在运行, 但没有响应。\\n可以在任务管理器里结束 LanShare.exe 后再试。");
189     ExitCode::from(1)
190 }
```



入口写好了

```
lanshare/
├── crates/
│   └── lanshare/
│       └── src/
│           ├── desktop/
│           │   └── mod.rs 这一集写的
│           ├── server/ 7 个文件
│           ├── cli.rs
│           ├── lib.rs 这一集写的
│           ├── logging.rs
│           ├── main.rs 这一集写的
│           └── net.rs 这一集写的
```

```
├── netinfo.rs
├── storage.rs
├── textboard.rs
├── tests/ 5 个文件
├── web/ 6 个文件
├── build.rs 这一集写的
├── Cargo.toml 这一集写的
├── lanshare-proto/ 13 个文件
├── lanshare-wasm/ 2 个文件
├── web-tests/ 2 个文件
└── Cargo.toml
```



编译、跑测试、看看帮助

```
Windows PowerShell

PS D:\code\lanshare> cargo build
  Updating crates.io index
  Locking 10 packages to latest Rust 1.96.0 compatible versions
  Adding equivalent v1.0.2
  Adding hashbrown v0.17.1
  Adding indexmap v2.14.2
  Adding serde_spanned v1.1.1
  Adding toml v1.1.6+spec-1.1.0
  Adding toml_datetime v1.1.1+spec-1.1.0
  Adding toml_parser v1.1.3+spec-1.1.0
  Adding toml_writer v1.1.2+spec-1.1.0
  Adding winnow v1.0.4
  Adding winresource v0.1.31
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 14.05s

PS D:\code\lanshare> cargo test -p lanshare --lib
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Finished `test` profile [unoptimized + debuginfo] target(s) in 4.10s
```



真实踩坑：入口里的三个小故事

🟢 let_ 和 let_lock：互斥量拿到就放掉了

带名字的才会活到作用域结束；编译器不会提醒

🟠 残留的实例记录：读到了上一次的旧端口

拿到互斥量后第一件事就删旧记录

🟠 测试环境也会骗人：Ctrl+C 怎么按都不退出

从 Git Bash 启动的进程，继承了“忽略 Ctrl+C”



选择题

为什么 `main.rs` 不用 `tokio main` 宏，而要手动建运行时？

- A 手动建的运行时更快
- B 主线程要留给托盘的消息循环，服务交给工作线程
- C `tokio main` 宏在 Windows 上不能用



选择题

为什么 main.rs 不用 tokio main 宏，而要手动建运行时？

A 手动建的运行时更快

速度不是原因，两种写法建出来的运行时是一样的。

C tokio main 宏在 Windows 上不能用

能用，只是我们的主线程另有用处。

B 主线程要留给托盘的消息循环，服务交给工作线程

消息循环会一直占着创建托盘的那个线程。我们让主线程干这件事，用户点退出、循环结束，main 就自然返回。用 tokio main 宏，主线程被运行时占着，就得另开线程跑托盘，结构更绕。



选择题

拿到互斥量之后，为什么第一件事就是删掉旧的实例记录？

- A 防止后来者读到上一次崩溃留下的旧端口
- B 为了节省磁盘空间
- C 因为 Windows 不允许覆盖文件



选择题

拿到互斥量之后，为什么第一件事就是删掉旧的实例记录？

A 防止后来者读到上一次崩溃留下的旧端口

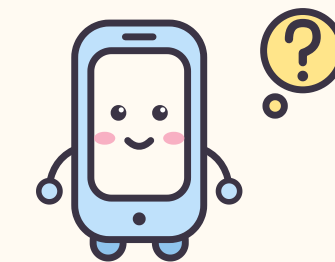
崩溃或断电时记录不会被删。拿到锁的实例先删旧记录、启动好了再写新的，后来者读到的就一定是当前这一个写的。

B 为了节省磁盘空间

一个实例记录只有几十个字节，不是为了省空间。

C 因为 Windows 不允许覆盖文件

写记录用的是先写临时文件再改名，和这个无关。



选择题

构建脚本里，为什么用 `CARGO_CFG_TARGET_OS` 环境变量判断，而不用 `cfg(windows)`?

- A 两个完全一样
- B 构建脚本跑在编译它的机器上，要问的是“编出来的程序给谁用”
- C `cfg` 在构建脚本里不能用



选择题

构建脚本里，为什么用 `CARGO_CFG_TARGET_OS` 环境变量判断，而不用 `cfg(windows)`？

A 两个完全一样

不一样。`cfg(windows)` 说的是构建脚本自己跑在哪个系统上。

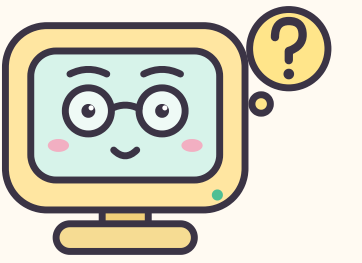
C `cfg` 在构建脚本里不能用

能用，只是问的问题不对。

B 构建脚本跑在编译它的机器上，要问的是“编出来的程序给谁用”

构建脚本在编译的那台机器上运行，`cfg(windows)` 回答的是那台机器。交叉编译时两者可能不同，所以要看 `cargo` 告诉我们的目标系统。

想一想



desktop 模块先放了一个替身，main.rs 就能先写完、先编译。这种“先替身、后实现”的做法，还在前面哪几集用过？它的前提是什么？



想一想

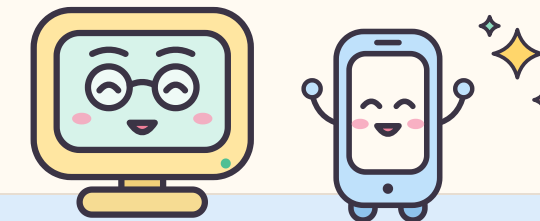
desktop 模块先放了一个替身，main.rs 就能先写完、先编译。这种“先替身、后实现”的做法，还在前面哪几集用过？它的前提是什么？

第 10 集的三个临时网页文件，第 2 集那个只打印一行的 main.rs，都是这个思路。前提是接口先定下来：名字、参数、返回值和真的一样，上层代码才能先写，换成真的时一个字都不用改。



这一集写了什么

- ✓ main.rs 是总调度：控制台、参数、单实例、绑端口、运行时、主界面、托盘、收尾
- ✓ 手动建 tokio 运行时，服务交给工作线程，主线程留给托盘
- ✓ net.rs：绑定预设 SO_EXCLUSIVEADDRUSE，被占就往后找，最多 10 个
- ✓ build.rs 在编译之前运行，把图标和版本信息编进 exe
- ✓ 先放替身，让上层代码先跑起来；下一集换成真的外壳



下一集

全课完结，感谢收看